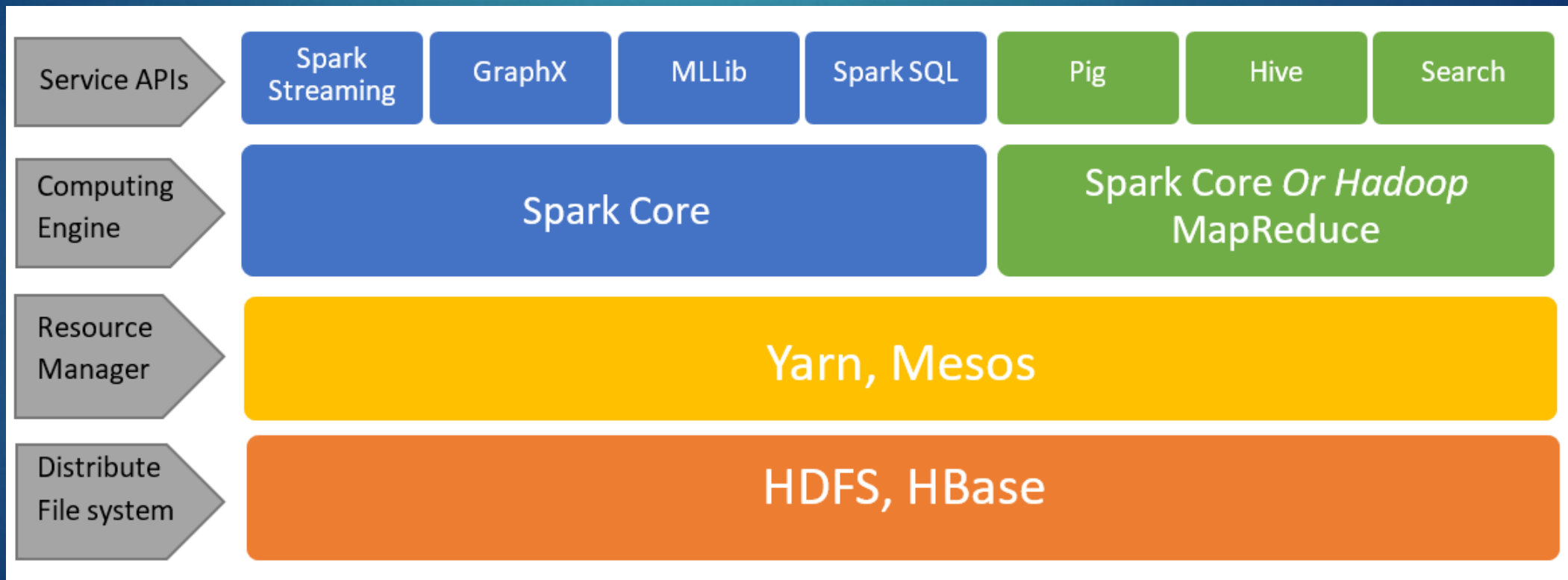


Setup Spark Cluster

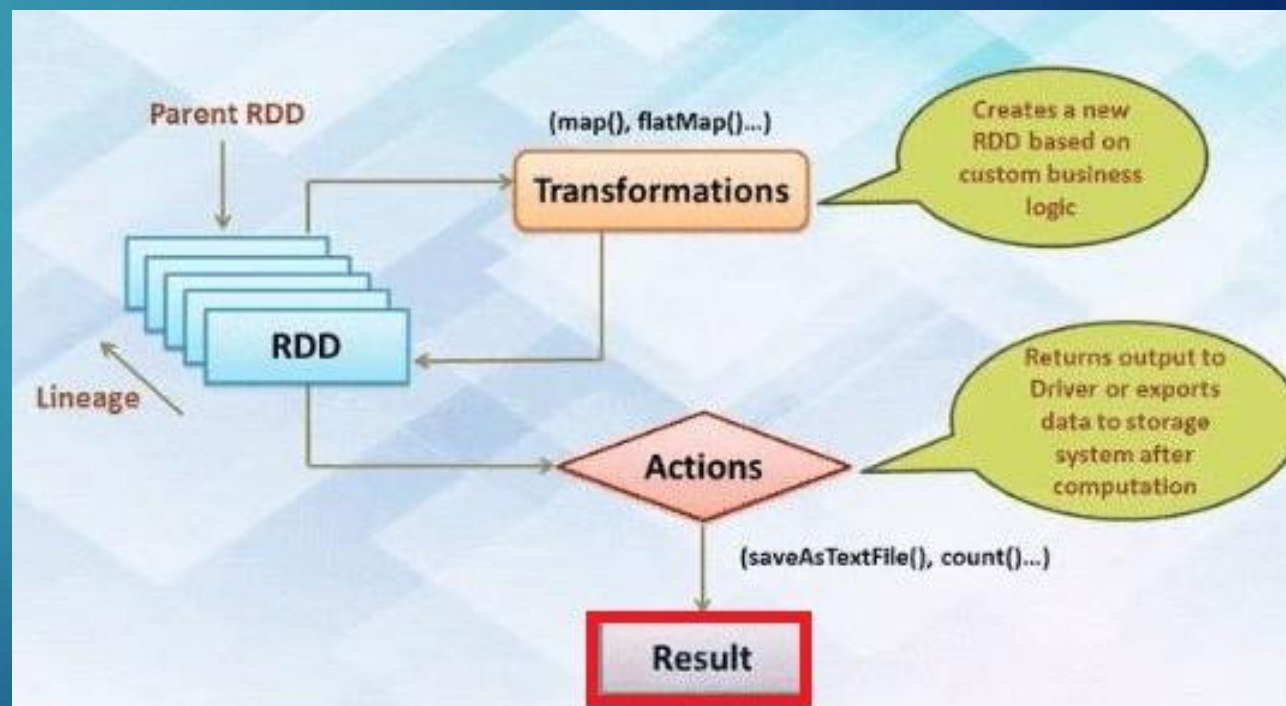
- ▶ First - Do **Assignments > Big Data > Setup EMR**
 - ▶ There is a small charge for this activity
 - ▶ Stop after slide #3 (which you click “Create Cluster”), you are done for now (no screen shots needed)
 - ▶ This will take several minutes to run, and we will return to this shortly



- ▶ Spark is an open-source, distributed processing system used for big data workloads
- ▶ The primary difference between Spark and Hadoop MapReduce is that Spark processes and retains data in memory for subsequent steps, whereas MapReduce processes data on disk



- ▶ Spark has 2 types of RDD operation: Transformations and Actions
- ▶ Transformations are functions that take an RDD as the input and produce one or many RDDs as the output
- ▶ Actions are operations to access the actual data available in an RDD and provide non-RDD values
- ▶ Transformation's output is an input of Actions





Big Data on the Cloud: Spark & Streaming

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



Big Data Streaming

7

- ▶ Big Data Streaming is a process in which large streams of real-time data are processed with the sole aim of extracting insights and useful trends out of it
- ▶ A continuous stream of unstructured data is sent for analysis into memory before storing it onto disk
- ▶ This happens across a cluster of servers and speed matters the most in big data streaming
- ▶ The value of data, if not processed quickly, decreases with time



Streaming vs. Batch

8



- ▶ Requires a set of data that is collected over time
- ▶ Handles a large batch of data
- ▶ Processes over all or most of the data
- ▶ Latency of the batch processing model will be in minutes to hours
- ▶ Lengthy process and is meant for large quantities of information that aren't time-sensitive

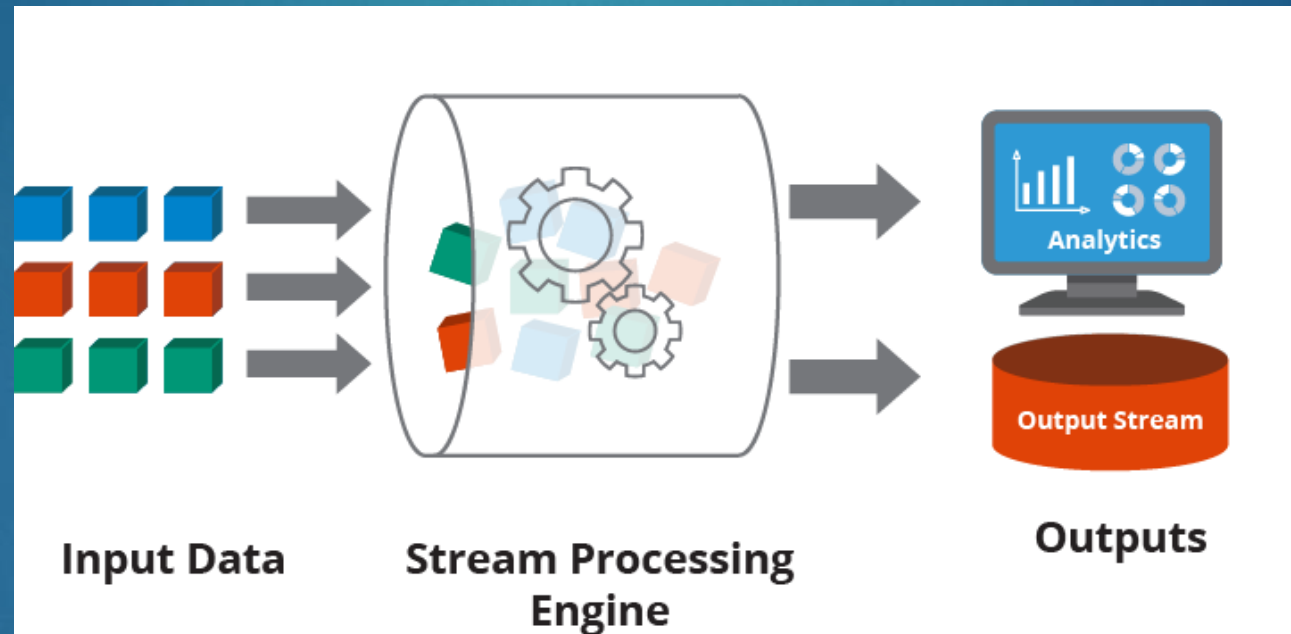


- ▶ Requires data to be fed into an analytics tool, often in micro-batches, and in real-time
- ▶ Handles individual records or micro-batches of few records
- ▶ Processes over data on a rolling window or most recent record
- ▶ Latency will be in seconds or milliseconds
- ▶ Processing is fast and is meant for information that is needed immediately

Streaming - Architecture

9

- Some examples of streaming data include IoT sensors, server and security logs, real-time advertising and click-stream data from apps and websites



- Data streams from one or more message brokers need to be aggregated, transformed and structured before data can be analyzed

Examples of Streaming Data

10

- ▶ A **financial firm** streams stock market data to detect sudden changes, recalculate risk exposure, and auto-adjust client portfolios within seconds
- ▶ A **delivery company** tracks its vehicles in real time using GPS and sensor data. The system reroutes drivers instantly to avoid traffic, reduce fuel usage, and meet delivery windows
- ▶ **Wearable health devices** stream patient vitals (e.g., heart rate, oxygen levels) to care providers. Alerts are triggered if readings exceed safe ranges, enabling immediate medical intervention
- ▶ A **weather company** ingests satellite and ground sensor data in real time to issue severe weather alerts faster and help emergency teams respond proactively
- ▶ A **utility provider** streams power consumption data from smart meters. It adjusts load and usage dynamically, helping to prevent blackouts and optimize energy distribution



Streaming Technologies

11

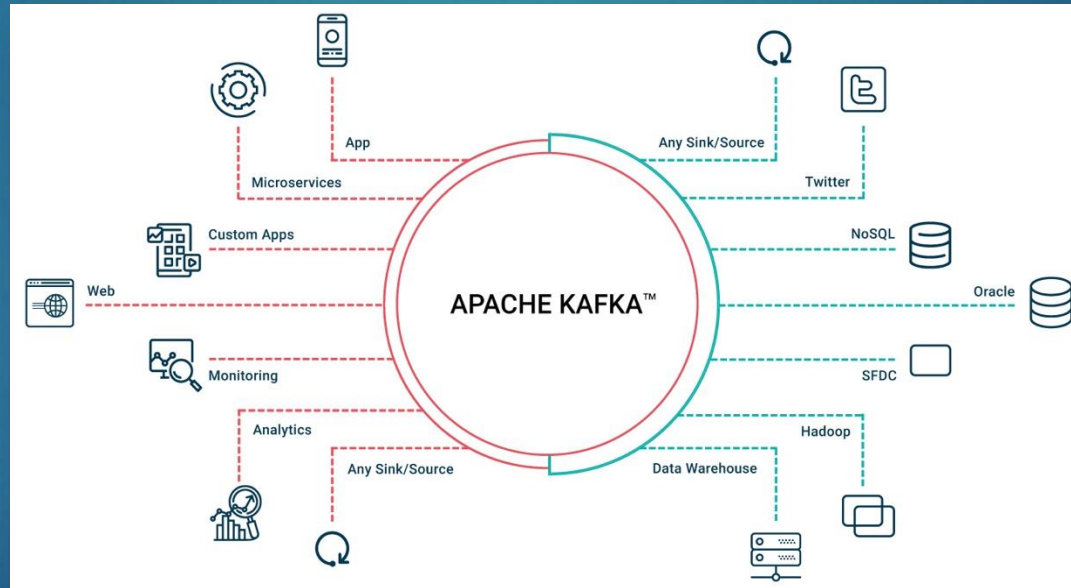
- ▶ Apache Kafka and Amazon Kinesis are platforms for durable, reliable, and scalable data ingestion
- ▶ Both share common concepts: producers, consumers, partitioning/sharding, and replication
- ▶ Designed for real-time processing, they can ingest thousands of simultaneous data feeds to support high-speed analytics and applications



Kafka

12

- ▶ Apache Kafka (originally developed at LinkedIn) is a distributed event store and stream-processing platform
- ▶ Its core architectural concept is an immutable log of messages that can be organized into topics for consumption by multiple users or applications



- ▶ It can be deployed on bare-metal hardware, virtual machines and containers (on-premise and cloud)

Kafka

13

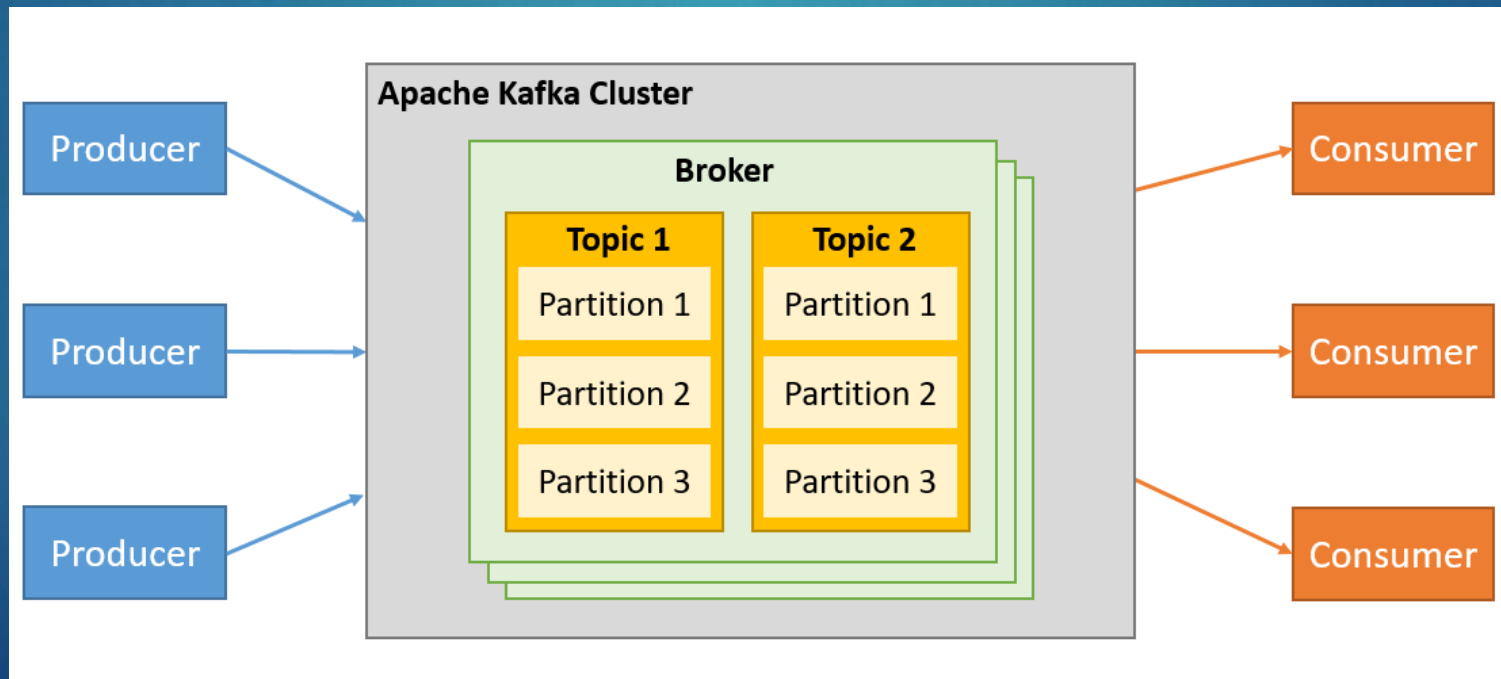
- Kafka is used by thousands of companies including over 80% of the Fortune 100



Kafka

14

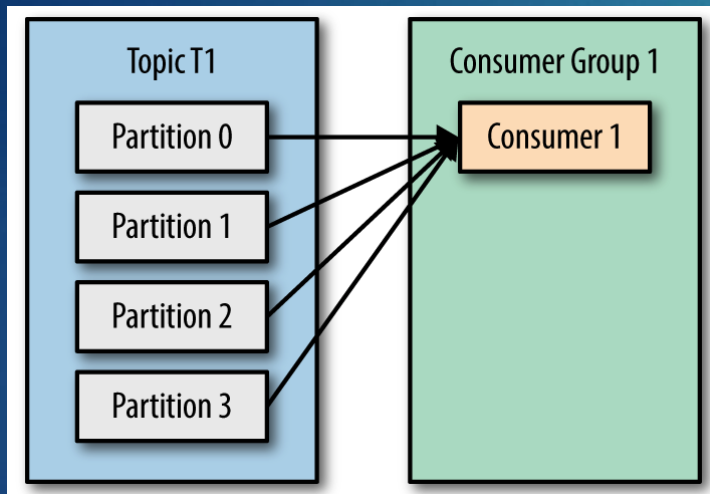
- ▶ Kafka stores messages in topics that are partitioned and replicated across multiple brokers in a cluster
- ▶ Producers send messages to topics from which consumers read
 - ▶ Within a topic, partitions can be added to improve scalability
- ▶ Consumers read messages from partitions within a topic



Kafka – Consumer and Consumer Groups

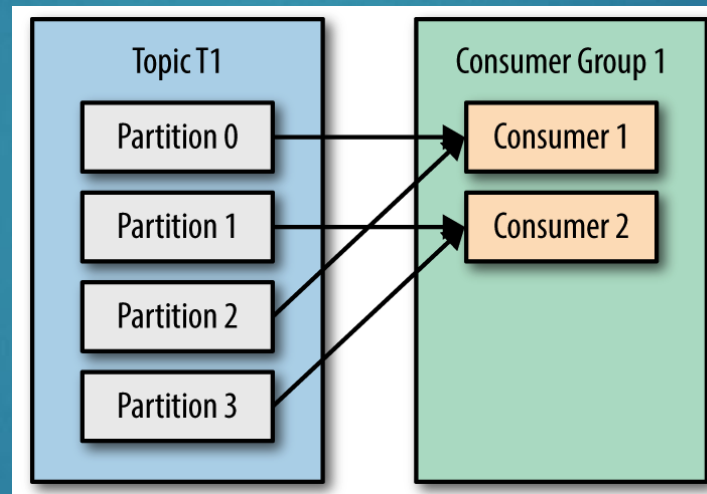
15

- ▶ Kafka Consumers are part of a Consumer group
- ▶ Question: What is a “Consumer” an example of?

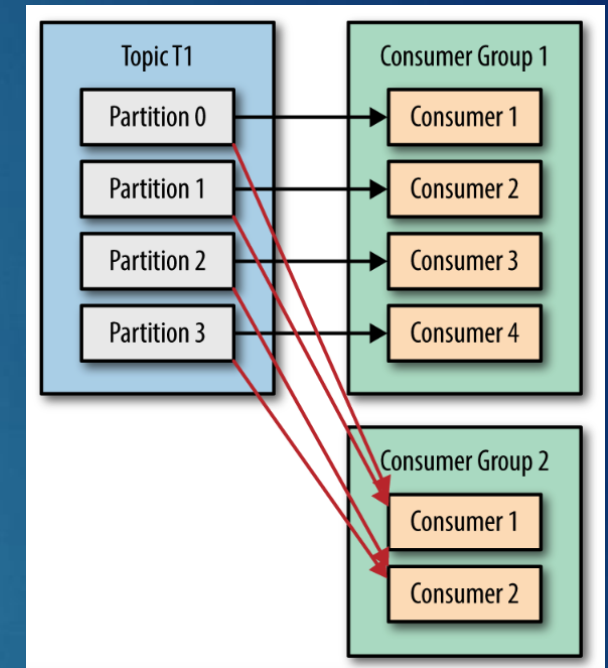


Consumer 1 will get all messages from all four T1 partitions

What happens if Consumer 1 cannot keep up with all the partitions?



Adding another Consumer to the group allows the 2 Consumers to consume the messages from the partitions more efficiently

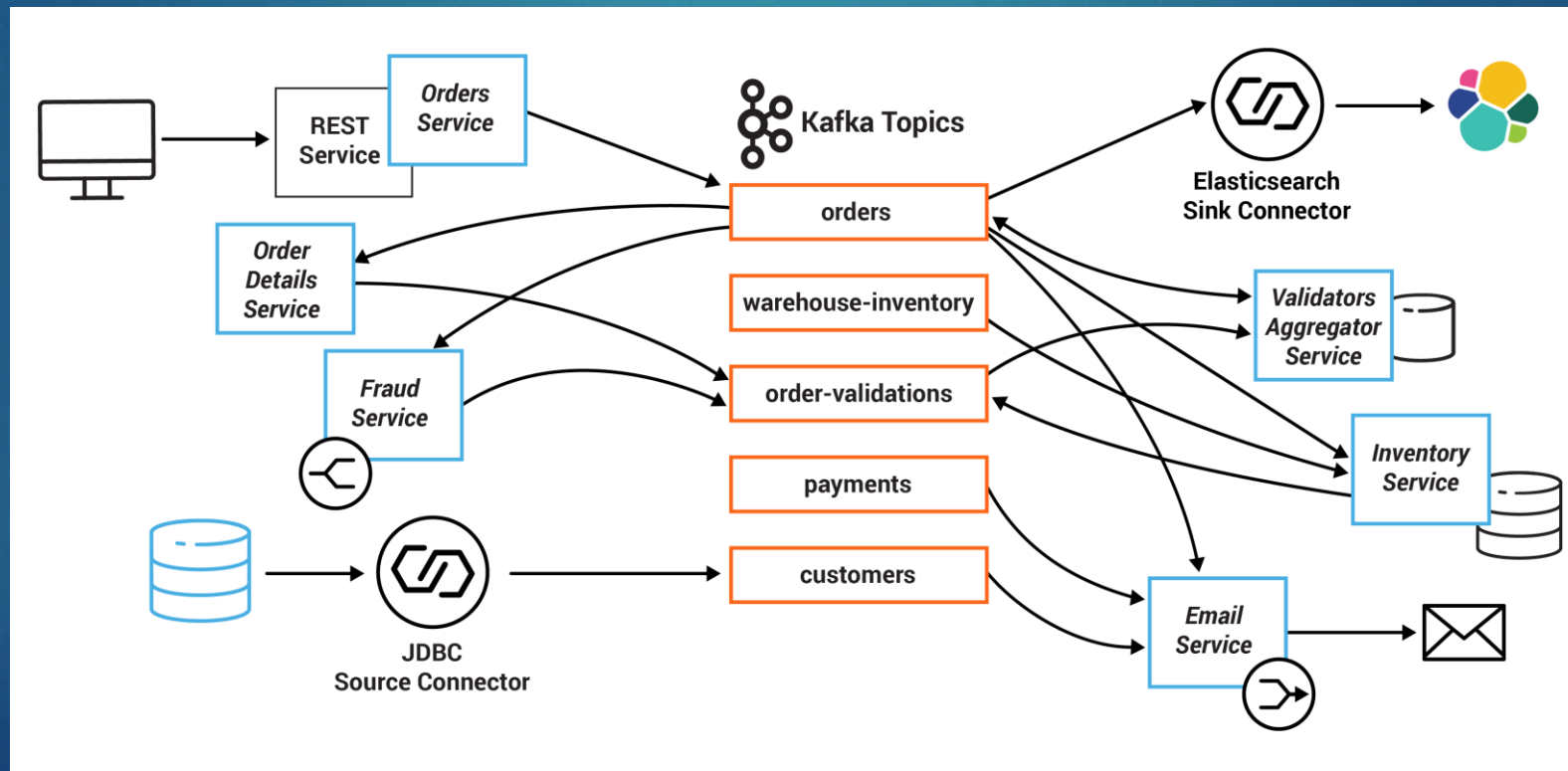


Two Consumer groups can independently process messages from the T1 partitions

Kafka Microservices Architecture - Example

16

- ▶ A system centers on an Orders Service which exposes a REST interface to POST and GET Orders
- ▶ Posting an Order creates an event in Kafka that is recorded in the topic orders
- ▶ This is picked up by different validation engines (Fraud Service, Inventory Service and Order Details Service), which validate the order in parallel, emitting a PASS or FAIL based on whether each validation succeeds



Kafka – Benefits

17

► Scalable Architecture

- Kafka's partition-based design distributes data across multiple brokers, enabling it to handle massive volumes of streaming data that go far beyond a single server's capacity

► High Performance & Low Latency

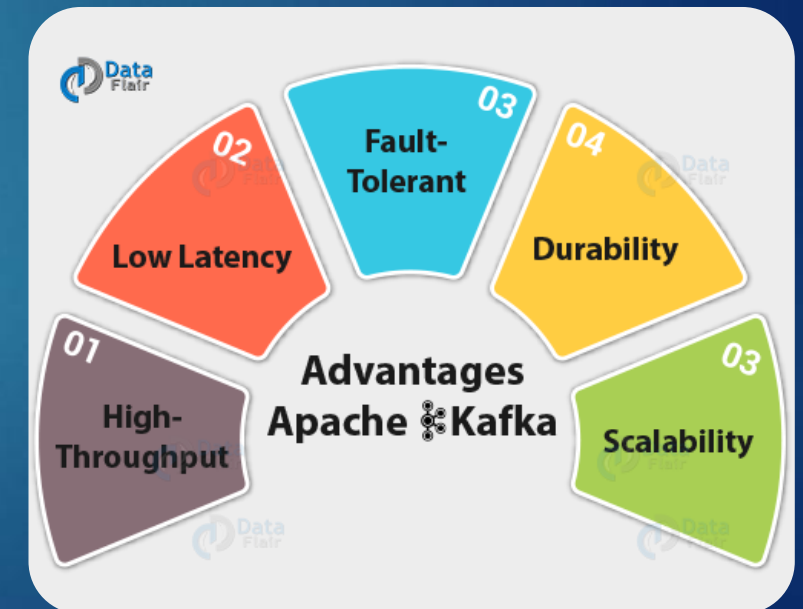
- Kafka decouples producers and consumers, enabling real-time data flow with minimal delay, making it ideal for high-throughput systems

► Durable & Fault-Tolerant

- Kafka is designed to withstand server failures through built-in replication and persistent storage, ensuring that data remains safe and available

► Reliable & Flexible Performance

- Kafka offers strong durability guarantees, scalable pub/sub and queue models, configurable consistency, and message ordering within partitions—all of which support stable, high-throughput applications



- ▶ Amazon Kinesis is a fully managed service that collects, processes, and analyzes real-time streaming data at any scale, enabling instant insights and actions from sources like IoT devices, clickstreams, and video feeds
- ▶ Key capabilities:
 - ▶ **Real-Time Streaming** – Process and analyze data as it arrives for immediate insights
 - ▶ **Scalable & Flexible** – Seamlessly handle any volume of data with pay-as-you-go pricing
 - ▶ **Faster Decisions** – Eliminate batch delays, accelerating automation and decision-making
 - ▶ **Diverse Sources** – Stream from video, audio, IoT sensors, application logs, and clickstreams
 - ▶ **Advanced Use Cases** – Power machine learning, anomaly detection, predictive analytics, and dashboards



Kinesis Capabilities

19



- ▶ Real-time
 - ▶ Enables ingesting data in real-time, so you can run analytics queries instantly without waiting for your data to accumulate to analyze in future
- ▶ Fully managed
 - ▶ You do not have to maintain servers or run any software upgrades
- ▶ Scalable
 - ▶ Automatically scales up and down and can handle any amount of incoming data

Kinesis Data Streams

20

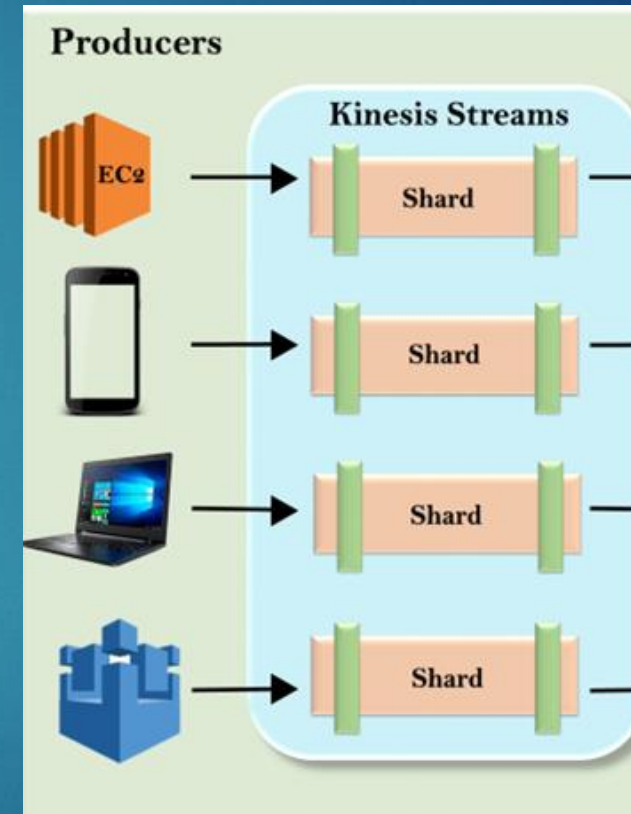
- ▶ Data Producers enter the records into Kinesis Data Streams (KDS)
- ▶ AWS offers the Kinesis Producer Library for simplifying producer application development



Kinesis Data Streams - Producers

21

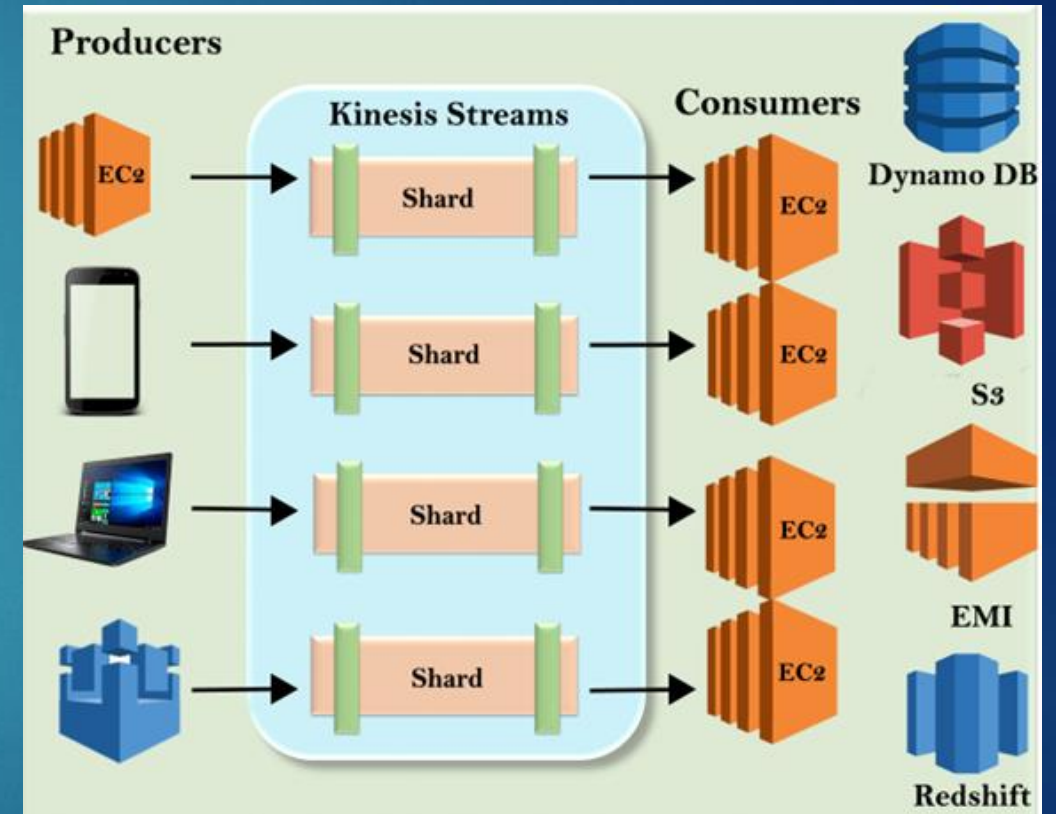
- ▶ Producers write records into a Kinesis Data Stream, which is composed of one or more shards.
- ▶ Each shard provides:
 - ▶ Writes: Up to 1,000 records per second (maximum 1 MB/sec)
 - ▶ Reads: Up to 5 transactions per second (maximum 2 MB/sec)
 - ▶ The capacity of a stream is determined by the number of shards you configure
 - ▶ The total throughput of the stream equals the combined capacity of all shards



Kinesis Data Streams - Consumers

22

- ▶ Consumers are applications that process data records from a Kinesis Data Stream
- ▶ Each consumer receives its own 2 MB/sec read throughput, independent of other consumers
- ▶ Multiple consumers can read from the same stream in parallel, without competing for read capacity



Kinesis or Kafka?

23

Service	Strengths	Challenges / Limitations	Best For
Apache Kafka	• Full flexibility & feature set • Latest version support • Highly customizable	• Requires significant setup & ongoing management • Higher DevOps overhead	Teams with strong in-house expertise needing maximum control
Amazon Kinesis	• Fully managed AWS service • Simple setup & integration • Scales easily with no infra management	• More limitations than Kafka • Less flexible for custom use cases	Teams that want easy streaming without managing infrastructure
Amazon MSK (Managed Streaming for Apache Kafka)	• Managed Kafka on AWS • Reduces operational burden • Uses native Kafka APIs	• Not always fully up to date with latest Kafka releases • Less flexible than self-managed Kafka	Teams that want Kafka features but prefer AWS to handle infrastructure

► In summary:

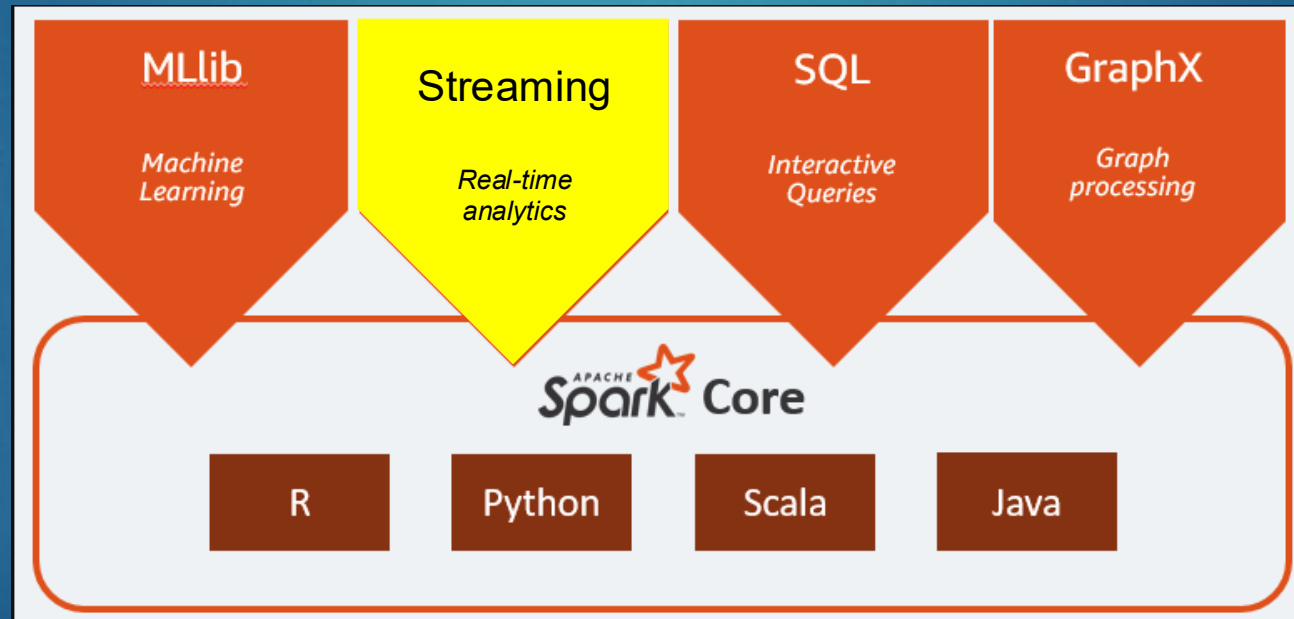
- Kafka → Control
- Kinesis → Simplicity
- MSK → Middle ground



Spark Streaming

24

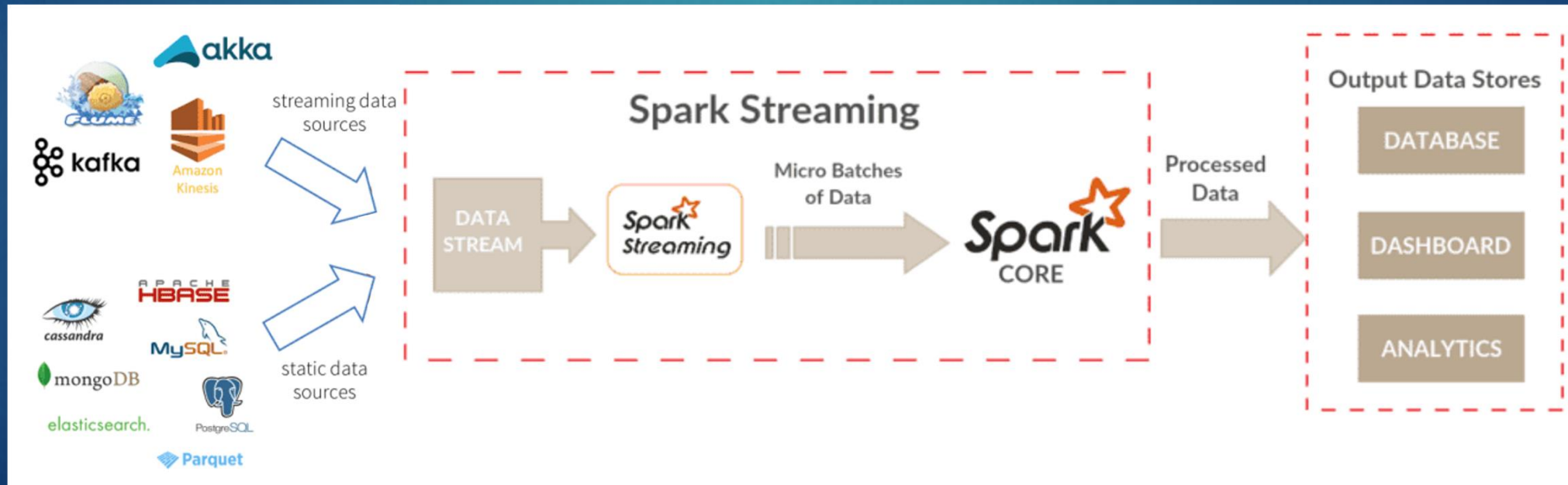
- ▶ Spark Streaming is an extension of the core Spark API that enables scalable, high-throughput, fault-tolerant stream processing of live data streams



Spark Streaming & Data Sources

25

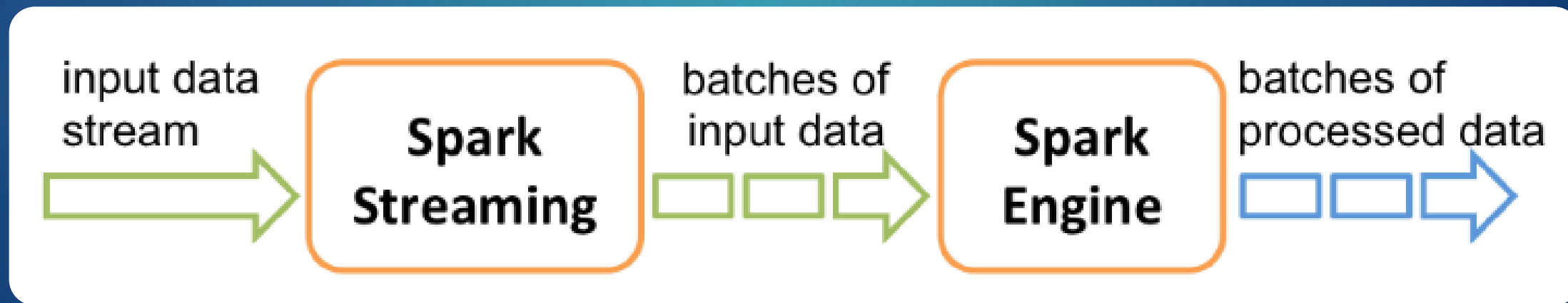
- ▶ Spark Streaming is an extension of the Spark API that enables real-time data processing for data engineers and data scientists
- ▶ It can ingest data from a wide range of sources, including Apache Kafka, Amazon Kinesis, and other streaming platforms
- ▶ The processed data can then be delivered to file systems, databases, or live dashboards to provide immediate insights



Spark Streaming – How does it work?

26

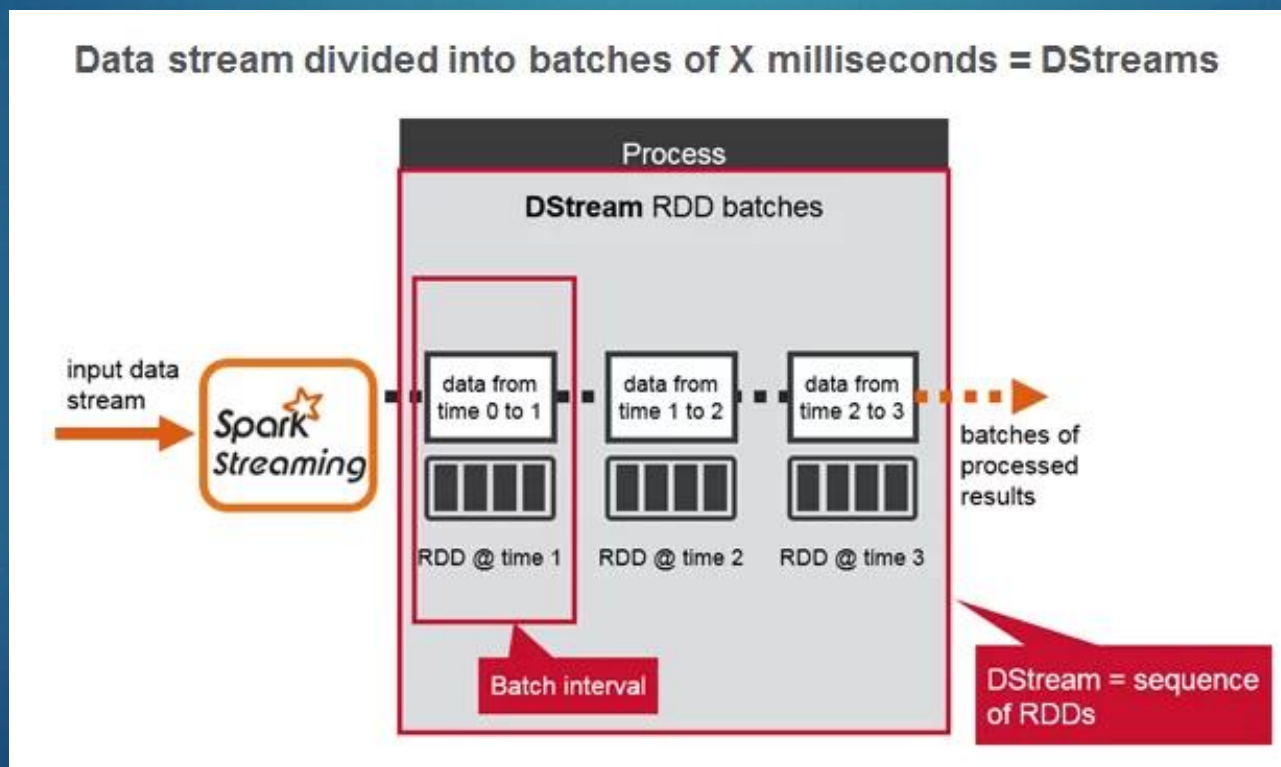
- ▶ **Real-Time Processing** – Ingests live data streams and processes them in small batches using the Spark engine
- ▶ **DStreams (Discretized Streams)** – A high-level abstraction that represents a continuous stream of data
- ▶ **RDD-based Architecture** – Each DStream is internally represented as a sequence of RDDs, enabling parallel and fault-tolerant processing



Spark Streaming – How does it work?

27

- ▶ Incoming data is grouped into batch intervals (based on time)
- ▶ Each batch interval generates a new RDD
- ▶ Every RDD contains the records collected during that specific interval



Spark Streaming – Simple example

28

► Wordcount.py - Streaming style

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext
```

```
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)
```

← Create a local StreamingContext and batch interval of 1 second

```
lines = ssc.socketTextStream("localhost", 9999)
```

← Create a **DStream** that will connect to hostname:port, like localhost:9999

```
wordCounts = lines.flatMap(lambda x: x.split(' ')) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda x,y: x + y)
```

← This is the same code from our previous wordcount example using an RDD

```
wordCounts.pprint()
```

← Print the first 10 elements of the RDD

```
ssc.start()           # Start the computation
ssc.awaitTermination() # Wait for the computation to terminate
```

← Loops waiting for events

► In a separate window, run Netcat (small utility found on most Linux/Unix systems)

```
nc -lk 9999
```

← Writing to port 9999

Spark Streaming – Simple example

29

Window #1 running Netcat

```
# TERMINAL 1:  
# Running Netcat  
  
$ nc -lk 9999  
  
hello world
```

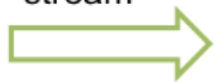
Window #2 running WordCount.py

```
$ spark-submit Wordcount.py localhost 9999  
...  
-----  
Time: 2014-10-14 15:25:21  
-----  
(hello,1)  
(world,1)
```



This can be
replaced by a
streaming
technology like
Kafka or Kinesis

input data
stream



**Spark
Streaming**

batches of
input data



**Spark
Engine**

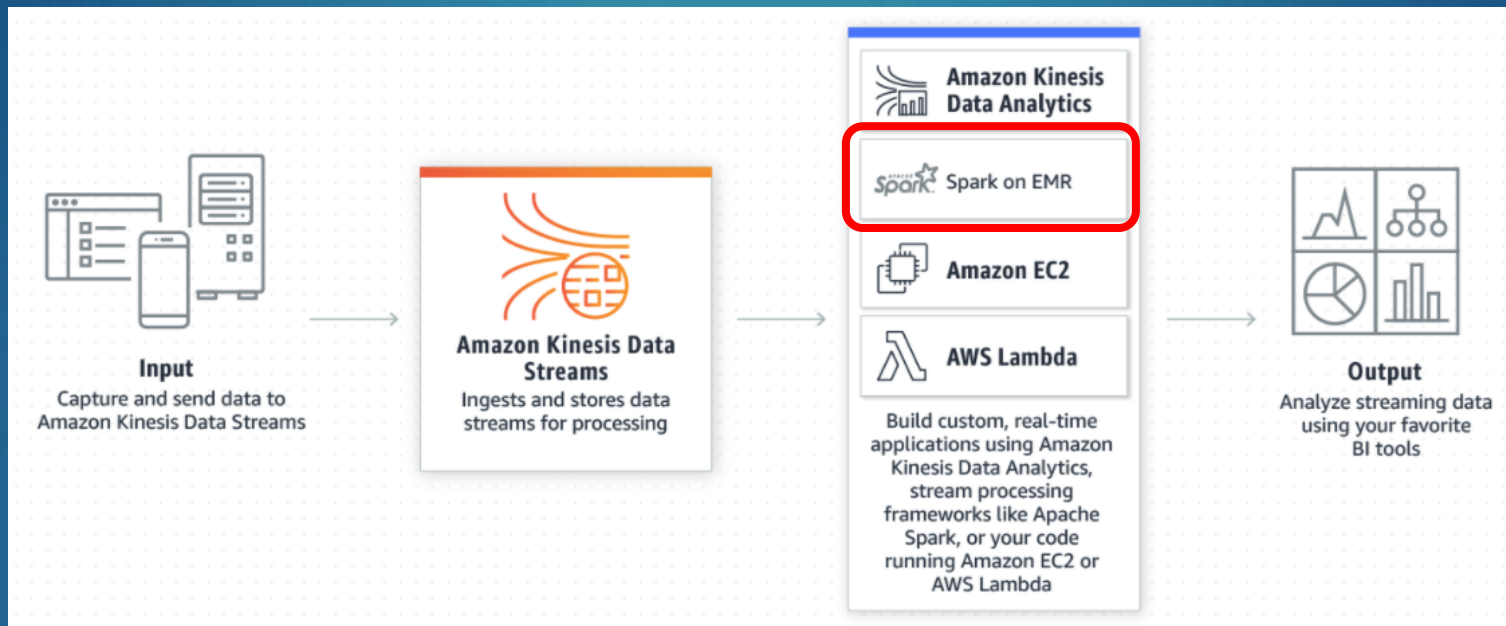
batches of
processed data



Kinesis Data Streams

30

- ▶ With Spark on EMR, you can create a DStream that connects to a Kinesis Data Stream, which can process the data as RDDs



Creates a **DStream**
from a Kinesis Data
Stream

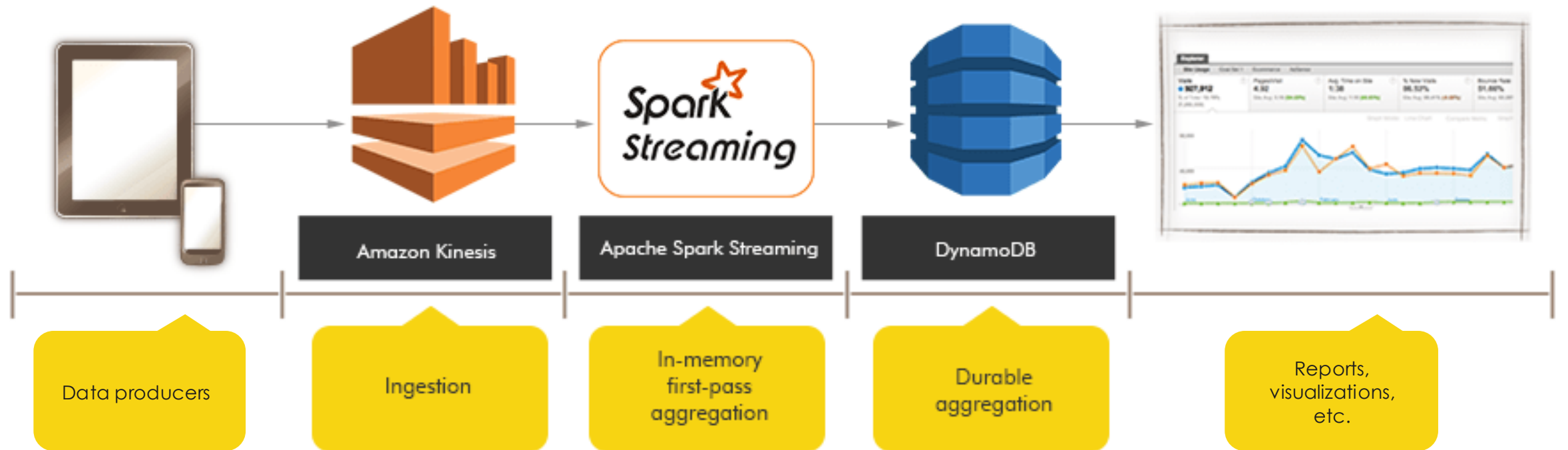
```
from pyspark.streaming.kinesis import KinesisUtils, InitialPositionInStream

kinesisStream = KinesisUtils.createStream( streamingContext, [Kinesis app name],
[Kinesis stream name], [endpoint URL], [region name], [initial position],
[checkpoint interval], StorageLevel.MEMORY_AND_DISK_2)
```


Kinesis + Spark Streaming

31

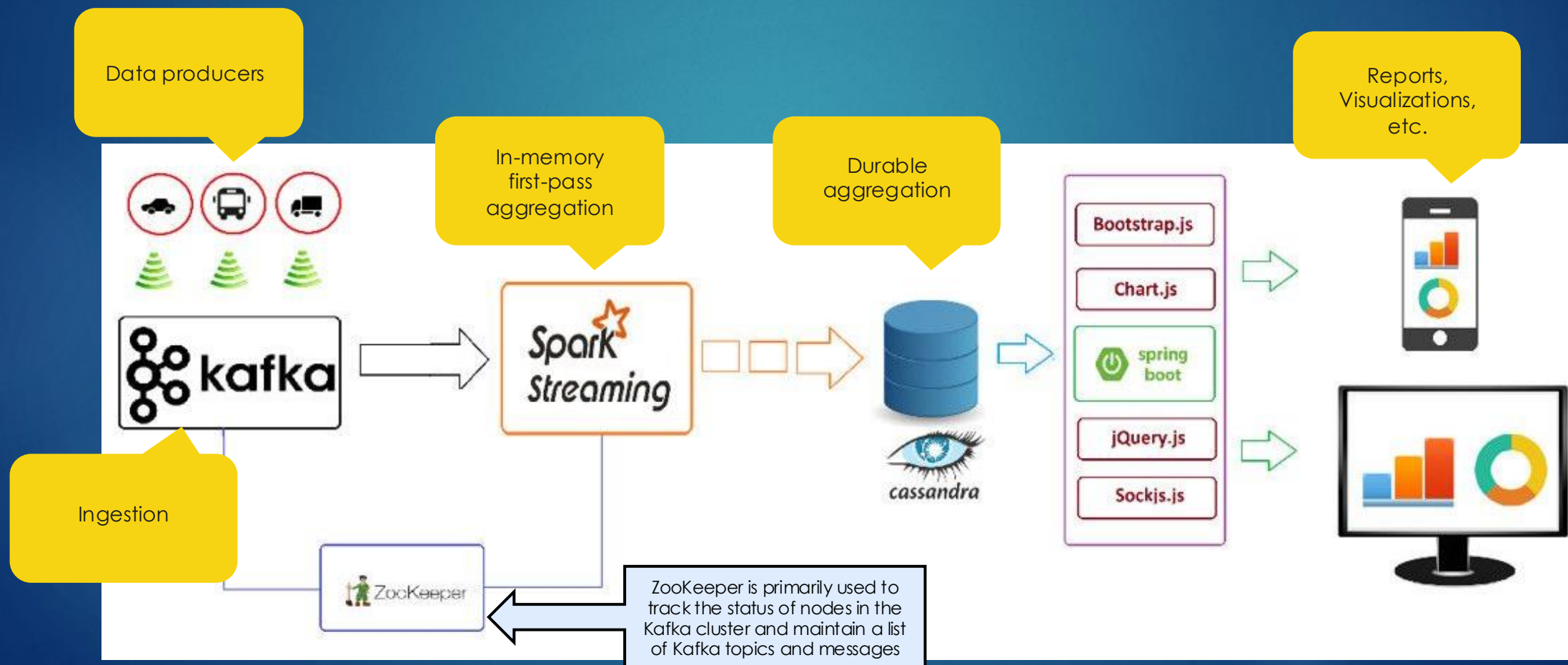
- Combining this all together



Kafka + Spark Streaming

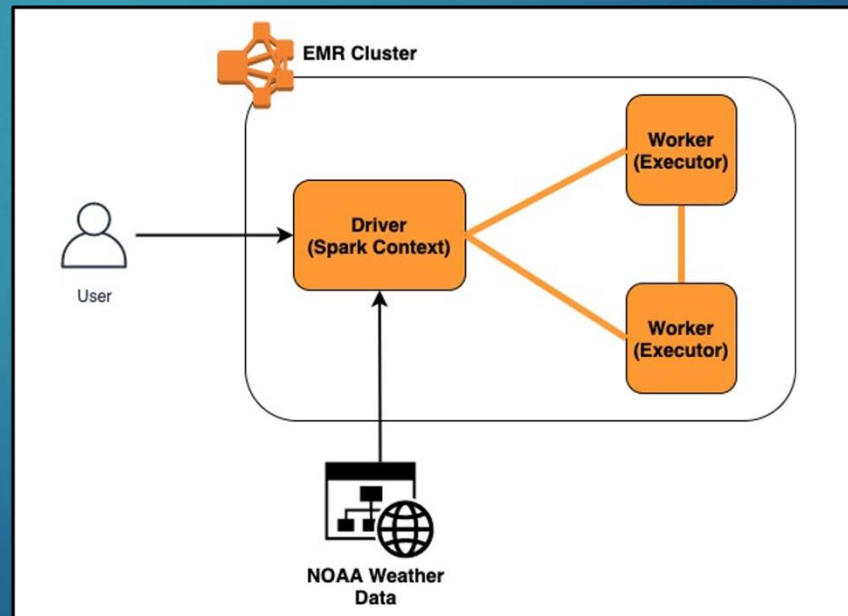
32

► Combining this all together



Weather modelling - Exercise

- ▶ This exercise goes beyond Wordcount and provides a more realistic use case using Spark
- ▶ We will use data from NOAA (National Oceanic and Atmospheric Administration)
 - ▶ This data is from Western-European weather stations from 1980 to 2014
- ▶ Follow the instructions in [Assignments > Activity - Evaluating Weather Data with Spark](#)
 - ▶ Part 1 due by end of class



- ▶ There are 3 deliverables plus an opportunity for a **bonus point** in this activity